

3. Project Design

FitBuddy — Architecture, components, data model, and interface design

System Architecture

FitBuddy uses a layered web architecture. The browser communicates with FastAPI routes. Routes validate inputs, coordinate CRUD operations, invoke the Gemini service wrapper, and render Jinja2 templates or JSON responses. SQLAlchemy provides persistence to SQLite.



Component Design

- `main.py`: creates the FastAPI application, initializes database metadata, mounts static files, and includes the router.
- `routes.py`: browser routes, JSON APIs, form handling, error responses, and orchestration.
- `schemas.py`: Pydantic validation models for user input and feedback.
- `database.py`: SQLAlchemy engine, declarative base, and database session dependency.
- `models.py`: User and Plan persistence models.
- `crud.py`: create/update/read/delete operations.
- `gemini_client.py`: central AI service and live/fallback behavior.
- `gemini_generator.py` and `gemini_flash_generator.py`: thin wrappers around AI generation roles.
- `updated_plan.py`: feedback-driven plan revision wrapper.
- `templates/`: user-facing pages; `static/css/style.css` controls visual presentation.
- `tests/`: API-level regression tests.

Data Model

User has a unique `user_id` plus `username`, `age`, `weight`, `goal`, `intensity`, `experience`, and `created_at`. Plan stores `user_id`, `original_plan`, `updated_plan`, `nutrition_tip`, `last_feedback`, `created_at`, and `updated_at`. The application retrieves the latest plan for a user using `created_at` ordering.

Interface Design

- Landing page: concise hero message plus plan-generation form.
- Result page: profile summary, seven-day plan, nutrition/recovery tip, and feedback form.
- Admin dashboard: users, profile details, original/updated plans, last feedback, and delete action.
- Responsive behavior: layouts collapse to one column on smaller screens.

Quick Reference

Layer	Technology	Responsibility
Presentation	Jinja2 + CSS	Forms, results, dashboard
Application	FastAPI	Routing and orchestration
Validation	Pydantic	Input constraints
Data	SQLAlchemy + SQLite	Persistent users/plans
AI	Google Gemini via service wrapper	Generation and revision
Testing	pytest + TestClient	Endpoint verification

Project basis: This document is derived from the supplied FitBuddy source package, including its README, FastAPI routes, SQLAlchemy models, Jinja2 templates, CSS, and tests.